

FUNDAMENTALS OF PROGRAMMING

Lecture 10: **MULTIDIMENSIONAL ARRAYS**

Instructor: Dr Aqib Perwaiz

DEPARTMENT OF COMPUTER ENGINEERING
COLLEGE OF E&ME

FUNDAMENTALS OF PROGRAMMING CS-114

Lecture 10

handing:

100 Students Name age Eng - - - -

Variables = $10 \times 100 = 1000$

500

1000 = 10,000 variables

Arrays

- An array is a collection of variables of similar type, placed contiguously in memory.
- Arrays are used to avoid the inconvenience of giving each data element a unique variable name.
- One data structure having more than one data members of the same type

Array Definition

- Array declaration tells the compiler about

- the type of array.

- the name of array.

- the size of array.

type arrayName [arraySize];

- Example:

float arr[20];

Array type

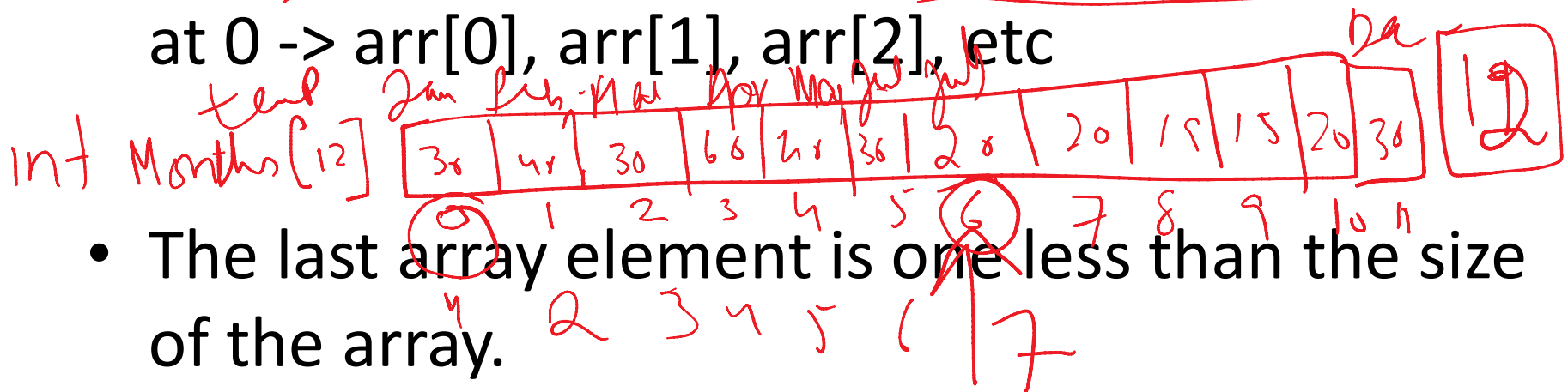
Array name

Array size

Handwritten notes:
Data type
float arr[20];
arr

Referring to Individual Elements in an Array

- All the array elements are numbered, starting at 0 -> `arr[0]`, `arr[1]`, `arr[2]`, etc



- The last array element is one less than the size of the array.

– If array size is 20, ie. `arr[20]` array subscripts would be from

`arr[0]` to `arr[19]`

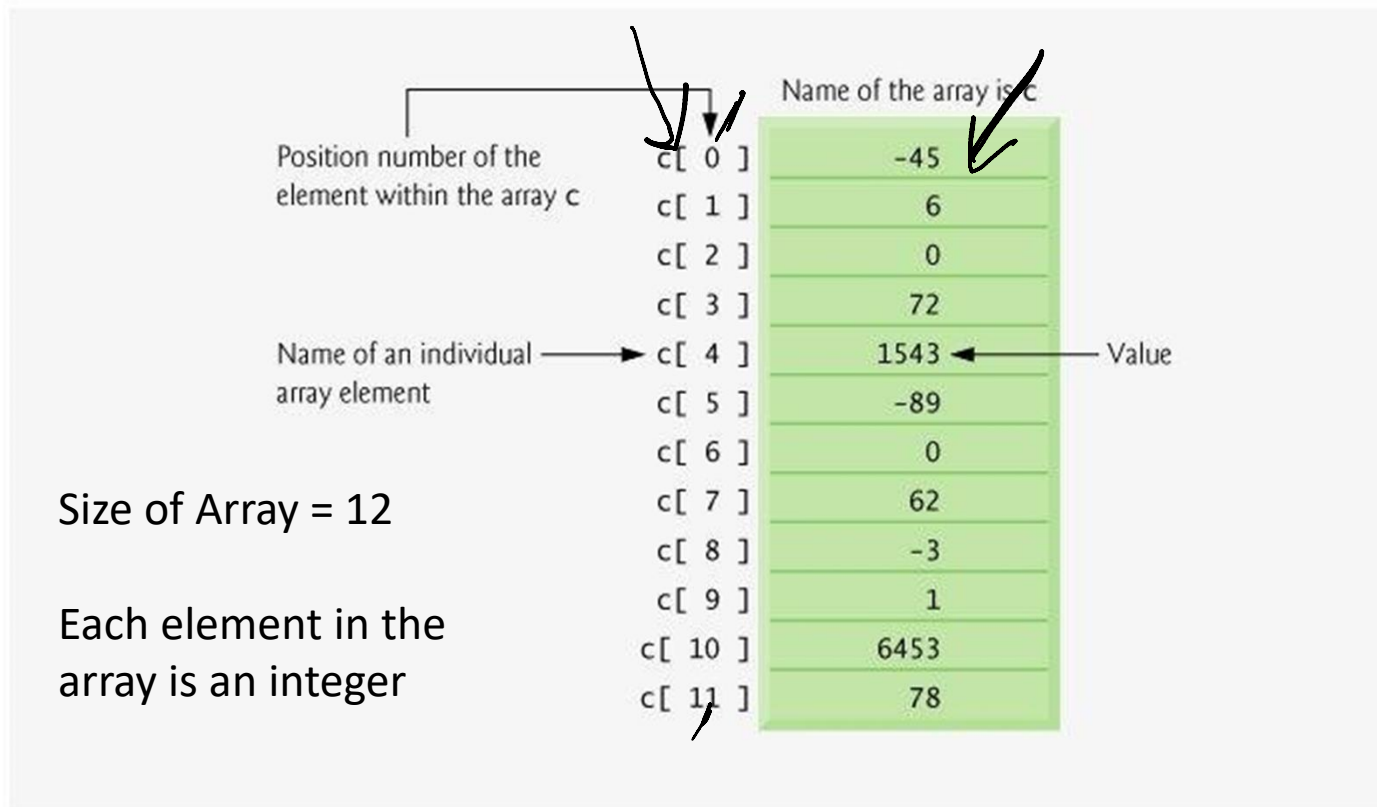
month temp[6]

Referring to Individual Elements in an Array

- Individual elements of the array are referred to with subscripts, the number in brackets following the array name
 - To use the 2nd element of the array -> `arr[1]`
 - To use the 1st element of the array -> `arr[0]`
 - To use the 9th element of the array -> `arr[8]`

Example

Integer Type Array



(Deitel and Deitel)

Initializing Arrays

rooms[1] ✓

- */* integer type array */*

int rooms [5] = { 50, 49, 20, 1, 17};

0



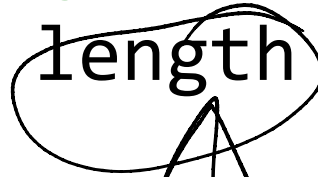
2

3

4

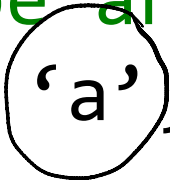
- */* float type array */*

float length [4] = { 12.39, 16.5,
15.3, 0.23};



- */* character type array */*

char name [4] = { 'a', 'A', 'z', 'Z'};



Initializing Arrays

- */*declared but uninitialized*

```
int n[10];
```

- */* initialize elements of array n with 0*

```
int n[ 10 ] = {};
```

```
int n[ 10 ] = { 0 };
```

*int n[10] = {0} same as
int n[10] = {0, ...}*

Initializing Arrays

- If the array size and an initializer list are specified in an array declaration and the number of initializers are greater than the array size.

- For example the array declaration

`int n[5] = { 32, 27, 64, 18, 95, 14 };`

- causes a **compilation error**, because there are six initializers and only five array elements.

`int n[5] = { 32, 27, 64, 18, 95 };`

Initializing Arrays

- If the array size and an initializer list are specified in an array declaration and the number of initializers is lesser than the array size.

- For example the array declaration

```
int n[5] = { 32, 27, 64 };
```

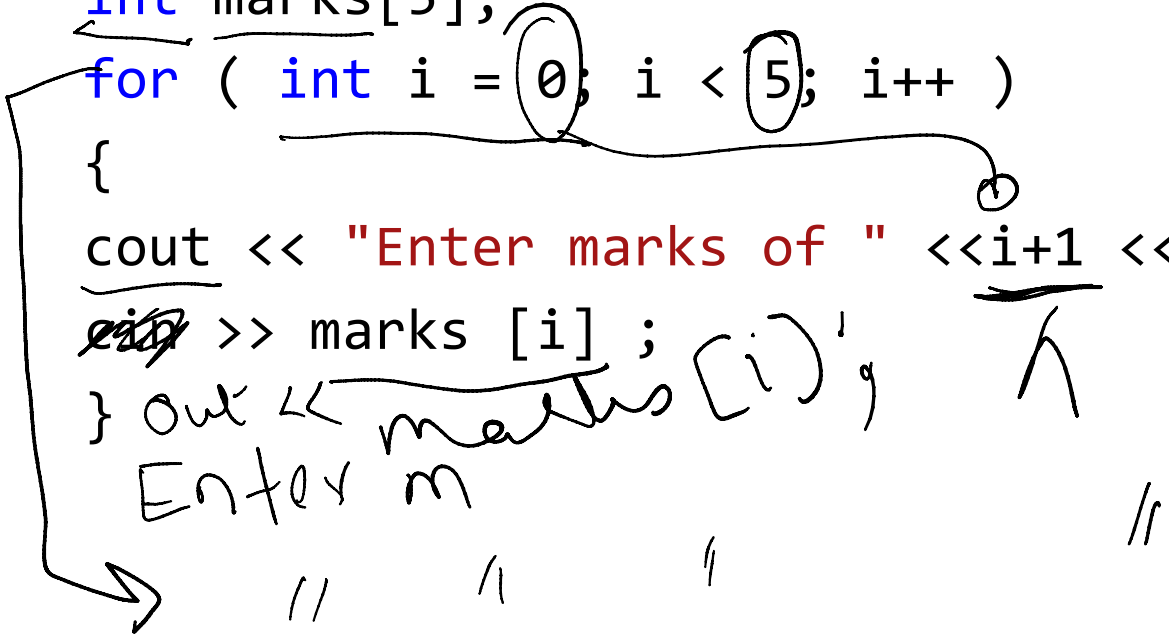
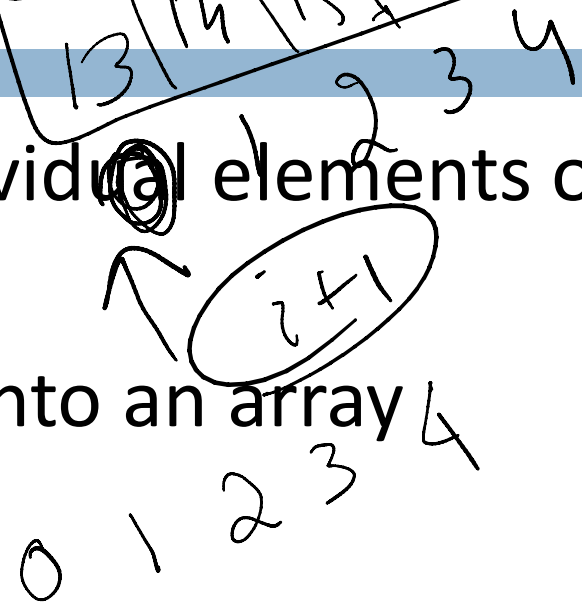
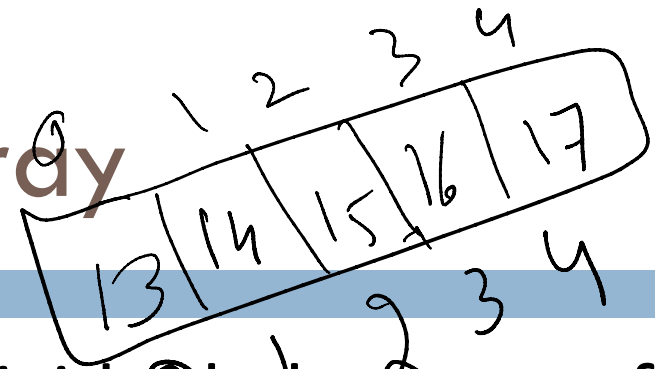
it will fill the remaining array positions with **0**

32	27	64	0	0
----	----	----	---	---

Entering Data into an Array

- Data can be entered into individual elements of an array using a **loop**
- Enter marks of five students into an array

```
int marks[5];  
for ( int i = 0; i < 5; i++ )  
{  
    cout << "Enter marks of " << i+1 << " student = ";  
    cin >> marks [i];  
} cout << marks[i];  
Enter m
```



Entering Data into an Array



- Enter marks of 1 student = 1
- Enter marks of 2 student = 1
- Enter marks of 3 student = 3
- Enter marks of 4 student = 4
- Enter marks of 5 student = 5

Entering Data into an Array

```
#include <iostream>
using namespace std;
int main(){
    int marks[5];
    for ( int i = 0; i < 5; i++ )
    {cout << "Enter marks of " <<i+1 <<" student = ";
    cin >> marks [i] ;}
    for ( int i = 0; i < 5; i++ )
    {cout <<  marks [i] ;
    }
}
```

Enter marks of 1 student = 5

Enter marks of 2 student = 4

Enter marks of 3 student = 6

Enter marks of 4 student = 3

Enter marks of 5 student = 8

54638

Reading Data from an Array

Reading data from the array "marks" and calculating the average.

```
sum = 0;  
for ( i = 0; i < 1050; i++ ) sum +=  
marks [i];  
cout<<"Average is "<<sum/1050<<endl;
```

Summing the Elements of an Array

```
□ int main()  
□ {  
□   const int Size = 10; //variable indicating  
□   size of array  
□   int a[Size] = {87,68,94,100,83,78,85,91,76,87};  
□   int total = 0;  
□   // sum contents of array a  
□   for ( int i = 0; i < Size; i++ ) total += a[ i ];  
  
□   cout << "Total of array elements: " << total <<  
□   endl;   return 0;  
□ }
```

Total of array elements: 849

Const variable

- Constant variables must be initialized with a constant value (or expression) when they are declared and cannot be modified thereafter.
- Constant variables are also called **read-only variables**.
- Only constant variables can be used to declare the size of arrays.

~~char~~ names[5] = {'A', 'b', 'c', 'D', 'E'}

Caution!

0 1 2 3 4

5

- Referring to an element outside the array bounds is an **semantic error**. It is not a syntax error.
 - i.e. it is not caught by the compiler
- ☐ When looping through an array, the array subscript should never go **below 0** and should always be **less than the total number of elements in the array** (one less than the size of the array).
- Make sure that the loop-termination condition prevents accessing elements outside this range.

count names[5];

Example

// Program to copy the contents of one array to another

```
int main()
```

```
{
```

```
int oldMarks[4] = {78, 92, 56, 67};
```

```
int newMarks[4];
```

```
for(int i=0; i<4; i++) newMarks[i] = oldMarks[i];
```

```
cout<< "New array is : "<< endl;
```

```
for(int i=0; i<4; i++)
```

```
cout<< "new Marks ["<<i<<" ] : "<<newMarks[i]<<endl;
```

```
return 0;
```

```
}
```

Sorting Algorithm - Bubble Sort

- In Bubble sort $N-1$ (N is Number of items to be sorted) passes are made through array.
- During 1st pass each element is compared with next one in the array and then their position is exchanged if required according to criteria (whether ascending or descending)



2 3 7 8 9

Sorting Algorithm - Bubble Sort

- At the end of 1st pass, the largest element gets to its proper position (in case of ascending order) i.e; the last location of array. So we do not compare it during 2nd pass. The 2nd pass is made like 1st one and at the end of 2nd pass, the 2nd largest element goes to its proper position. So we do not compare it during 3rd pass and so on it continues.

Bubble Sort Algorithm

Procedure BubbleSort(arr[1..n])

{ Bubble sort algorithm to arrange elements in ascending order }

for j ← n to 2 do

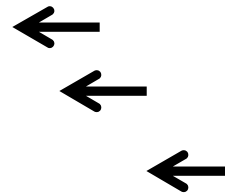
for i ← 1 to (j-1) do

if arr[i] > arr [i+1] then

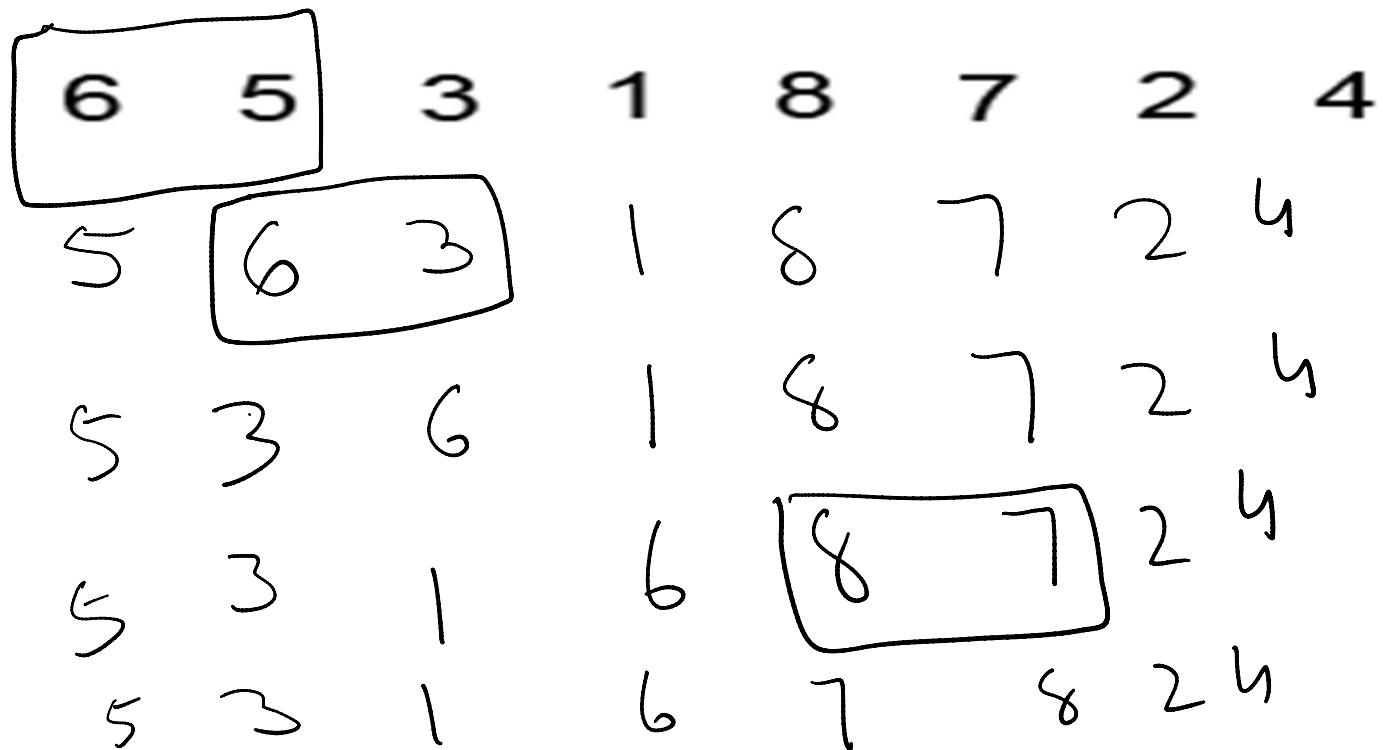
 hold ← arr[i]

arr[i] ← arr[i +1]

arr[i +1] ← hold



Bubble Sort Examples



Example

Given array is:

22 91 35 78 10 8 75 99 1 67
1 8 10 22 35 67 75 78 91 99

```
#include <iostream>using namespace std;
int main() { int a[10] = { 22, 91, 35, 78, 10, 8, 75, 99,
1, 67 };
    cout<<"Given array is:"<<endl;
    for (int i = 0; i < 10; i++)
        cout<< a[i] <<" ";
    cout<<endl;
    int temp,i,n;
    for (int i = 0; i < 9; i++)
    {
        for (int j = i + 1; j < 10; j++)
        {
            if (a[j] < a[i])
            {
                temp = a[i];
                a[i] = a[j];
                a[j] = temp;
            }
        }
    }
    for (int i = 0; i <
10; i++)    cout<< a[i] <<" ";
    return 0;}
```

Handwritten annotations:

- Blue arrows pointing to indices i and j above the array.
- Blue box around the first row of the array: 22 91 35 78 10 8 75 99 1 67.
- Red box around the comparison logic: $a[j] < a[i]$, $a[i] = 35$, $temp = 35$, $a[j] = 35$.
- Red circle around the value 91 in the array.
- Red circle around the value 35 in the array.
- Red circle around the value 10 in the array.
- Red circle around the value 1 in the array.
- Red circle around the value 67 in the array.
- Red circle around the value 105 in the array.
- Red circle around the value 105 in the array.
- Red circle around the value 105 in the array.

Multi Dimensional Arrays (Matrices)



- An array can have two or more dimensions. This permits them to model multidimensional objects, such as graph paper or the computer display screen.
- They are often used to represent **tables of values** consisting of information arranged in **rows** and **columns**.

Multi Dimensional Arrays (Matrices)



- To identify a particular element, we must specify two subscripts. By convention,

array_name[row_number][col_number]

- For example,

`arr[i][j]`

Multiple-Subscripted Arrays

- Multiple subscripted arrays
 - Tables with rows and columns (m by n array)
 - Like matrices: specify row, then column

	Column	Column 1	Column	Column 3
Row 0	a[0][0]	a[0][1]	a[0][2]	a[0][3]
Row 1	a[1][0]	a[1][1]	a[1][2]	a[1][3]
Row 2	a[2][0]	a[2][1]	a[2][2]	a[2][3]

Array name Row subscript Column subscript

6.9 Multiple-Subscripted Arrays

- `int a[3][4];`

	Column	Column 1	Column	Column 3
Row 0	<code>a[0][0]</code>	<code>a[0][1]</code>	<code>a[0][2]</code>	<code>a[0][3]</code>
Row 1	<code>a[1][0]</code>	<code>a[1][1]</code>	<code>a[1][2]</code>	<code>a[1][3]</code>
Row 2	<code>a[2][0]</code>	<code>a[2][1]</code>	<code>a[2][2]</code>	<code>a[2][3]</code>

Array name `a` points to the first element of the first row.
 Row subscript points to the first element of the first row.
 Column subscript points to the first element of the first row.

6.9 Multiple-Subscripted Arrays

- Initialization

- `int b[2][2] = { { 1, 2 }, { 3, 4 } };`
- Initializers grouped by row in braces

`b[0][0]`

`b[0][1]`

1	2
3	4

`b[1][0]`

`b[1][1]`

6.9 Multiple-Subscripted Arrays

- Initialization

- `int b[2][2] = { { 1, 2 }, { 3, 4 } };`
- Initializers grouped by row in braces

1	2
3	4

- If not enough, unspecified - **elements set to zero**

1	0
3	4

`int b[2][2] = { { 1 }, { 3, 4 } };`

- Referencing elements

- Specify row, then column
`cout<<b[0][1];`

Initializing a 2D Array

- `int matrix [10] [2] = { {59 , 78},{14 , 17},{68 , 28},{32 , 45},
{ 5 , 14},{12 , 15}, {6, 2}, { 22,1},
{14,16},{2, 58} };`
- The values used to initialize an array are **separated by commas and surrounded by braces**.

2-D Array (Matrix)

```
int matrix [4] [4] = { {59, 78, 14, 17}, {68, 28, 32, 45},  
                      {5, 14, 12, 15}, {6, 2, 22, 1} };
```

(row = 2, col = 0) →

59 (0,0)	78 (0,1)	14 (0,2)	17 (0,3)
68 (1,0)	28 (1,1)	32 (1,2)	45 (1,3)
5 (2,0)	14 (2,1)	12 (2,2)	15 (2,3)
6 (3,0)	2 (3,1)	22 (3,2)	1 (3,3)

4 x 4 Matrix

Reading a 2D Array

```
int matrix [4][4]={ {59,78,14,17}, {68,28,32,45},  
                    {5, 14,12,15}, {6,2, 22, 1}};
```

```
for(int i=0;i<4;i++)  
{  
  for(int j=0;j<4;j++)  
  {  
    cout<<matrix[i][j]<<"\t";  
  }  
  cout<<endl;  
}
```

Example

- Write a matrix with following entries

12	12
54	34
16	3
12	6
43	23
1	2

2D String

1001	A	l	i	\0					
1011	A	b	b	a	s	\0			
1021	A	l	i	n	a	\0			
1031	A	y	e	s	h	a	\0		
1041	B	a	n	o	\0				

2D array of characters

```

1  /* Fig. 6.21: fig06_21.c
2      Initializing multidimensional arrays */
3  #include <stdio.h>
4
5  void printArray( const int a[][ 3 ] ); /* function          */
   prototype
6
7      /* function main begins program execution */
8  int main()
9  {
10     /* initialize array1, array2, array3 */
11     int array1[ 2 ][ 3 ] = { { 1, 2, 3 }, { 4, 5, 6 } };
12     int array2[ 2 ][ 3 ] = { { 1, 2, 3, 4, 5 } };
13     int array3[ 2 ][ 3 ] = { { 1, 2 }, { 4 } };
14
15     cout<<"Values in      by row are:\n" ;
   array1
16     printArray( array1 );
17
18     cout<<"Values in      by row are:\n" ;
   array2
19     printArray( array2 );
20
21     cout<<"Values in      by row are:\n" ;
   array3
22     printArray( array3 );
23     return 0; /* indicates successful termination */
24
25 } /* end main */
26
27

```

```

28  /* function to output array with two rows and three columns */
29  void printArray( const int a[][ 3 ] )
30  {
31      int i; /* counter */
32      int j; /* counter */
33
34      /* loop through rows */
35      for ( i = 0; i <= 1; i++ ) {
36
37          /* output column values */
38          for ( j = 0; j <= 2; j++ ) {
39              cout<<a[ i ][ j ] ;
40          } /* end inner for */
41
42          cout<<"\n" ; /* start new line of output */
43      } /* end outer for */
44
45  } /* end function printArray */

```

Program Output

```

Values in array1 by row are:
1 2 3
4 5 6
Values in array2 by row are:
1 2 3
4 5 0
Values in array3 by row are:
1 2 0
4 0 0

```

Examples Using Arrays

- Character arrays

Its is used in programming for storing and manipulating text, such as words, names and sentences.

- Character arrays can be initialized using string literals

```
char string1[] = "first";
```

- Null character '\0' terminates strings

- string1 actually has 6 elements

- It is equivalent to

```
char string1[] = { 'f', 'i', 'r', 's', 't', '\0' };
```

- Can access individual characters

string1[3] is character 's'

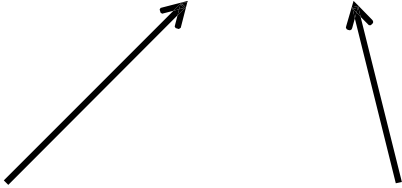
Character arrays

- Like all other variables and arrays

```
char name[5];  cin >> name;  
cout<< "you entered " << name;
```

- The << operator displays the output until it encounters a null character.
- In case of a string variable, the cin operator places the null automatically.

Multi-Dimensional Strings (Array of Strings)

- char stu_lists [5] [10] = {"Ali",
"Abbas",
"Alina",
"Ayesha",
"Bano",
};
- 
- The diagram consists of two black arrows. The first arrow starts at the first element of the array, 'Ali', and points diagonally up and to the right. The second arrow starts at the second element of the array, 'Abbas', and points diagonally up and to the left. These arrows illustrate the mapping between the array elements and the strings in the list.

Passing Elements of an Array to a Function



- Arrays are passed
 - By Reference

Passing Arrays to Functions

- Passing arrays

- To pass an array argument to a function, specify the name of the array without any brackets

```
int myArray[ 24 ]; myFunction( myArray, 24 );
```

- Array size usually passed to function
 - Arrays passed **call-by-reference**
 - Name of array is address of first element
 - Function knows where the array is stored
 - Modifies original memory locations

- Passing array elements

- Passed by call-by-value
 - Pass subscripted name (i.e., myArray[3]) to function

Passing Arrays to Functions

- Function prototype

```
void modifyArray( int b[], int arraySize );
```

- Parameter names optional in prototype only

- `int b[]` could be written `int []`
- `int arraySize` could be simply `int`

- The following are equivalent prototypes:

```
void modifyArray( int b[], int arraySize );
```

```
void modifyArray( int [], int );
```


Acknowledgements

1. Deitel and Deitel: C++ How to Program, 7th Edition, Prentice Hall Publications
2. Robert Lafore: Object-Oriented Programming in C++, Fourth Edition, December 2001, Sams Publishing .
3. A Structured Programming Approach Using C++ by Behrouz A. Forouzan
4. www.cplusplus.com